ADS-C in the OPDI trajectory

Version 1 — what a satellite cruise feed adds to flight-event extraction

EUROCONTROL Performance Review Unit — Open Performance Data Initiative

2026-10-17

OPDI builds every track from ADS-B state vectors. OpenSky also publishes an ADS-C feed, and the obvious hypothesis is that merging the two improves coverage and therefore milestone extraction. This paper tests it. ADS-C is merged into the state-vector frame *before* segmentation, so `track_id` sees both sources; the trajectory cleaner is made source-aware, because its twenty-second isolated-point rule would otherwise delete a feed whose median cadence is ten minutes. Both arms — ADS-B only, and ADS-B plus ADS-C — are rebuilt from the same state vectors with the same segmentation and the same detectors, and scored against EUROCONTROL APDF over the twenty aerodromes and three days of flight events version 4.

The feed is narrow and the result is correspondingly narrow. Measured over eleven days spanning 2023-09 to 2026-10, ADS-C delivers a few thousand rows a day inside OPDI's European box at a median of several hundred seconds between reports; almost all of those rows are above FL300 and most sit west of 10°W. It carries no `on_ground` flag and almost nothing below ten thousand feet. The A-CDM runway family therefore cannot move, and this paper reports that as a measured null rather than omitting it.

The two feeds also do not share a clock, and neither states a position age that can be used. Comparing ADS-C reports against ADS-B interpolated to the same instant separates the two candidate explanations: the feeds agree about where the track is to within a few metres across track while disagreeing by hundreds of metres along it, which is a timing offset and not a position error. That offset is a property of the individual airframe rather than of the feed, so no global correction is applied and the paper says why.

Every figure is produced by `benchmarks/regenerate_adsc_v1.py`. Where an output is not staged the table says so in place rather than being omitted.

Table of contents

1 What the feed is 4

2 How it is merged 7

3 Whether the two clocks agree 8

4 What the merge did to the trajectories 9

5 What it did to the flight events 10

6 What this does not measure 11

7 Reproducing this 12

8 Provenance 13

1 What the feed is

OpenSky publishes an ADS-C topic beside the ADS-B archive: `s3a://readsb-aircraft-states/topics/read` hourly partitions named `hour=<unix seconds>`, holding snappy **parquet** despite the `.avro` in the topic name — the files carry the Avro schema `org.opensky.avro.readsb.Aircraft` in their footer. Partitions run from 2023-07-07 to the current hour, 982 days with 24 gaps longer than a day, and in 2026 the feed averages 22.3 of 24 hourly partitions per day. A missing hour is routine, and the ingest tolerates it by listing before reading rather than letting `spark.read.parquet` raise `PATH_NOT_FOUND` on the first absent one.

Table 1.1: The raw ADS-C topic, by sampled day. Every band after `In box` is counted **inside** OPDI's published European coverage box – that is the population the pipeline would ingest. `Hours read` matters: a day with 6 of 24 partitions present has a cadence that reflects the gap, not the feed.

Day	Hours read	Rows	In box	Airframes in box	Median cadence (s)	> FL300	< 10,000 ft	West of 10°W	On ground	Exact duplicates
2023-09-01	10	1182	352	203	832.000	344	3	235	0	0
2024-03-01	9	409	140	97	953.000	138	1	117	0	0
2024-09-02	9	1310	438	242	832.000	432	4	308	0	0
2025-03-01	11	1030	322	203	1139.000	306	4	249	0	0
2025-09-01	24	271393693		902	494.130	3643	15	3094	0	0
2026-03-22	6	4210	284	74	277.200	268	6	187	0	0
2026-06-01	24	231164973		941	566.000	4870	48	3706	0	0
2026-06-05	24	278915420		994	543.000	5298	67	4134	0	0
2026-06-06	24	270985066		979	568.097	4963	44	3896	0	0
2026-06-07	24	257144725		953	564.645	4576	63	3553	0	0
2026-10-15	24	322065314		885	540.341	5122	52	3731	0	0

Reading that table across the archive: the feed delivers 409-32206 rows a day worldwide and 140-5420 inside the coverage box, from 74-994 in-box airframes. On the three study days it is 4725-5420 in-box rows from 953-994 airframes, at a median of 543-568 seconds between reports

from one aircraft. **Against the 5 s resolution OPDI ingests ADS-B at, that is a feed roughly two orders of magnitude sparser.**

It is also a *cruise* feed, and that is what bounds what it can contribute. Of the in-box rows, 94.4-98.6% are above FL300 and 65.8-83.8% sit west of 10°W — the North Atlantic corner of the box. The whole box sees 1-67 rows a day below ten thousand feet, and **`on_ground` is true for 0 rows across every day sampled.** There is also no redelivery to clean up: 0 exact duplicates in total, so the ingest's deduplication has never had anything to remove.

Table 1.2: Every column of the feed, by the best fill rate it reached on any sampled day. A column at 0.0 here was empty on **every** day sampled – which is the evidence for not carrying it into the OPDI schema.

Column	Best fill rate on any sampled day (%)
time	100.0
hex	100.0
lat	100.0
lon	100.0
alt_baro_ft	100.0
on_ground	100.0
track	99.9
flight	98.1
baro_rate	82.3
gs	82.2
squawk	50.7
alt_geom	0.0
ias	0.0
tas	0.0
mach	0.0
mag_heading	0.0
true_heading	0.0
roll	0.0
track_rate	0.0
geom_rate	0.0
oat	0.0
tat	0.0
wd	0.0
ws	0.0
nav_qnh	0.0
nav_altitude_mcp	0.0
nav_altitude_fms	0.0
nav_heading	0.0
registration	0.0
ac_type	0.0
ac_desc	0.0
category	0.0

That table is the reason this study carries five fields and not twenty-five. A reader would reasonably expect an ADS-C report to bring indicated airspeed, Mach, outside air temperature, wind, the selected altitude from the FMS, and the aircraft's registration and type. The `readsb` record has a column for each of them, and **every one measured 0.0% on all eleven days**.

They are not discarded by OPDI; they are not there. The same is true of `alt_geom`, so an ADS-C sample contributes barometric altitude and no geometric altitude at all. `nic`, `rc`, `seen`, `seen_pos` and `rssi` are present but constant — zero, zero, zero, zero and −49.5 — which makes them defaults rather than measurements, and they are not carried as if they were.

Table 1.3: Fill rate by day for the columns that are sometimes populated. The feed's schema has **grown**: two step changes, neither announced anywhere.

Day	track	gs	baro_rate	flight	squawk	on_ground
2023-09-01	0.0	0.0	0.0	0.0	0.0	0
2024-03-01	0.0	0.0	0.0	0.0	0.0	0
2024-09-02	0.0	0.0	0.0	0.0	0.0	0
2025-03-01	0.0	0.0	0.0	0.0	0.0	0
2025-09-01	99.9	65.2	65.3	0.0	0.0	0
2026-03-22	99.9	82.2	82.3	0.0	0.0	0
2026-06-01	99.7	58.0	58.0	97.7	44.1	100
2026-06-05	99.8	61.6	61.6	97.0	46.2	100
2026-06-06	99.8	61.9	61.9	98.0	50.7	100
2026-06-07	99.7	60.7	60.7	97.0	49.8	100
2026-10-15	99.9	71.3	71.3	98.1	37.8	100

Two step changes are visible there, and neither is documented upstream. `track`, `gs` and `baro_rate` are empty until somewhere between 2025-03 and 2025-09; `flight`, `squawk` and `on_ground` until between 2026-03 and 2026-06. Before roughly 2025-09 the feed is position and barometric altitude and nothing else. For this study that is harmless — the three study days are fully populated, and the callsign is deliberately dropped in any case — but **a merged arm over 2023 or 2024 would write ADS-C rows with NULL `velocity` and NULL `vert_rate`**, and any speed-based detector would see NULL rather than a value on those rows. If the study period ever moves back, that moves with it.

2 How it is merged

Decision	Value	Where
Merge point	Before segmentation	`TrackProcessor._merge_adsc`
Default	Off (`adsc.enabled = False`)	`AdscConfig`
ADS-C callsign	`NULL`	`osn_adsc.to_opdi_schema`
Shadow window	−60 s … +60 s	`AdscConfig.shadow_{before,after}_seco`
Global time correction	0 s (off)	`AdscConfig.time_offset_seconds`
Thinning applied to ADS-C	None	`AdscIngestion.ingest_from_s3`
Isolated-point rule	Exempt for ADS-C	`CleaningConfig.sparse_sources`
Stale-broadcast rule	Exempt for ADS-C	`CleaningConfig.sparse_sources`
Dedup tie-break	ADS-B wins	`drop_duplicate_statevectors`
Derivative-spike filter	Unchanged	`mask_derivative_spikes`
Segment split	Unchanged, 300 s	`CleaningConfig.segment_gap_seconds`

Four of those need an argument rather than an assertion.

Why before segmentation. The hypothesis is about coverage and track continuity, and both are properties of `track_id` — which `_add_track_id` assigns from the frame it is handed. Merging afterwards could only attach ADS-C to tracks ADS-B had already drawn, which answers a different question: *which ADS-B tracks can ADS-C be attached to*, rather than *what tracks do the two feeds make together*. A test reads that ordering off the source of `process_month` so it cannot drift.

Why the callsign is dropped. The feed's `flight` field carries the IATA commercial number — `LH133`, `AF805`, `AY68` — and ADS-B broadcasts the ICAO callsign, `DLH133`. The production segmentation arm (A9, `debounced`) breaks a track when the last non-blank callsign changes, so carrying `flight` through would trip that break at every ADS-C sample and split precisely the tracks the merge exists to join. A9 bridges NULLs by design, with `F.last(..., ignorenulls=True)`, so NULL removes a false signal rather than losing a true one. An IATA-to-ICAO mapping would recover the field; it was not attempted here.

Why the cleaner had to change. `CleaningConfig.isolated_max_gap_seconds` is 20 s, and `mask_isolated_points` NULLs any value whose nearest same-column neighbour is further away than that on *both* sides. At a cadence of several hundred seconds that is every ADS-C sample, in every column it touches. Left unexempted, step 02a would delete the position, altitude and speed of the entire feed it had just been handed — and the merged arm would then score identically to the baseline for a reason having nothing to do with the hypothesis. A test asserts exactly that failure with the exemption switched off, so the result cannot be quietly reintroduced.

Why `track_id` changes, and what that costs. Enabling the merge is a fourth `track_id` boundary after legacy → A8 → A9. Like those it is silent to a joiner — an empty join, not an error — and that is why both arms of this study live under `s3a://eurocontrol/opdi/research/adsc/` and the flag ships off. No published table moves.

3 Whether the two clocks agree

ADS-B and ADS-C arrive over different links and neither states a position age that can be used. ADS-B's `last_pos_update` is real but small. ADS-C's `seen` and `seen_pos` are constant zero — the feed *asserts* that every report is current — and the `readsb` record does not expose the ADS-C report's own time-of-position, so there is nothing in band to correct from. The skew was therefore measured.

For each ADS-C report with an ADS-B sample of the same airframe within 60 s on **both** sides, the ADS-B position is interpolated to the ADS-C timestamp and the offset decomposed against the direction the ADS-B pair actually moved — not against the broadcast heading, which in wind is not where the aircraft is going. **Cross-track** is what the two feeds disagree about that no timing explanation can absorb; **along-track divided by ground speed** is the implied time offset. A position error is isotropic and lands in both; a timing error lands only along track.

*`adsc_time_sync_2026.csv` is not staged, so this section's figures are absent rather than w

*`adsc_time_sync_pairs_2026.csv` is not staged, so this section's figures are absent rather

*`adsc_time_sync_2026.csv` is not staged, so this section's figures are absent rather than w

`the time-sync outputs` is not staged, so this section's figures are absent rather than wrong. Run `python benchmarks/regenerate_adsc_v1.py` with cluster credentials; the Provenance chapter lists what is present.

One thing the skew does **not** reach: the derivative-spike filter. `CleaningConfig.latlon_d1_max_deg_s` is 0.01 deg/s, about 1,100 m/s of apparent ground speed, and a few hundred metres of displacement across a several-hundred -second gap adds of the order of 0.001 deg/s. The filter is three orders of magnitude away from seeing this error, so it neither corrects it nor is damaged by it — which is also why no source guard was added there. Adding one would have been machinery for a risk that is not present.

4 What the merge did to the trajectories

*`adsc_gain_2026.csv` is not staged, so this section's figures are absent rather than wrong.

*`adsc_gain_2026.csv (extension row)` is not staged, so this section's figures are absent ra

*`adsc_holes_2026.csv` is not staged, so this section's figures are absent rather than wrong

adsc_gain_2026.csv is not staged, so this section's figures are absent rather than wrong. Run python benchmarks/regenerate_adsc_v1.py with cluster credentials; the Provenance chapter lists what is present.

5 What it did to the flight events

*`adsc_events_2026.csv` is not staged, so this section's figures are absent rather than wron

*`adsc_events_by_airport_2026.csv` is not staged, so this section's figures are absent rathe

Whatever those tables show, one thing cannot be read out of them, and it is worth stating before they are read at all: **the A-CDM and runway families could not have moved.** ADS-C has no `on_ground` flag that is ever true, the whole coverage box sees 1-67 of its rows a day below ten thousand feet, and its cadence is several hundred seconds against detectors that interpolate a runway-threshold crossing between adjacent samples. A flat row for `ATOT`, `ALDT`, `AOBT` or `AIBT` is the expected outcome of adding this feed, not a failure of the detector or of the measurement. The families where a change is physically possible are the ones anchored at the edges of coverage — `first_seen`, `last_seen`, the flight-level crossings, and ADEP/ADES through the flight list for North Atlantic arrivals.

And for any event that *did* move, the move is not automatically an improvement. Nothing in the pipeline prevents an event being interpolated between an ADS-B and an ADS-C sample, so an event derived across that boundary inherits the along-track displacement measured in the previous chapter. See the limitations below.

6 What this does not measure

Each of these is a measurement that was not made, not a result:

- **Only Europe.** Scoring is inside `EUROPE_BBOX`, and most of the in-box ADS-C rows sit in its Atlantic corner. The feed is global; a worldwide arm would see far more of it, and APDF could not score it.
- **Only June 2026, three days.** The flight-events-v4 period, chosen for comparability. ADS-C volumes in 2023–2024 are not scored, and the feed carried fewer columns then.
- **No shadow-window sweep.** −60/+60 s is a reasoned default, not a tuned one. A wider window drops more ADS-C; a narrower one admits near-duplicate positions. At the measured skew a symmetric window already catches every duplicate pair, so the asymmetry is what makes the time base explicit rather than a fix for a present failure.
- **No per-airframe time correction, and no stability study of the skew.** The time-synchronisation chapter measures the offset on three days and a few tens of airframes. Whether an airframe's offset is stable across weeks — the precondition for correcting it — is not measured here.
- **No IATA-to-ICAO callsign recovery.** The feed's `flight` field is discarded rather than mapped, so ADS-C contributes no callsign evidence to segmentation or to the flight list.
- **Nothing stops an event being interpolated across a source boundary.** `segment_id` splits at 300 s coverage holes and is read by **no detector** — `grep -rn segment_id src/opdi/pipeline/` finds it in two comments. An event derived between an ADS-B and an ADS-C sample therefore inherits the along-track displacement. This is pre-existing: the same detectors already interpolate across thirty-minute ADS-B holes. Making `segment_id` load-bearing would change published detector behaviour, so it is named here as a follow-up rather than changed on this branch.
- **The MLAT topic is untouched.** `readsb.aircraft.mlat.avro` is a second unused OpenSky source, denser than ADS-C — of the order of 374 objects a day — but starting only at 2026-05-18. It is multilateration, not ADS-C, and is out of scope.
- **No measurement of ADS-C positional accuracy.** `nic` and `rc` are constant zero in the feed, so the usual integrity containment bounds are unavailable. Nothing here establishes how accurate an ADS-C position is, only what including it changes — and the cross-track figure bounds the *disagreement*, which is not the same thing as an error.

7 Reproducing this

```
# 1. The two arms. ~6 cluster-hours. Not regenerated by the entrypoint.
#    Day-major, not step-major: step 01 keeps a three-day sliding window and
#    drops what the window has passed, so ingesting every day before
#    segmenting any of them would hand day 05 a window two-thirds absent.
export PYSPARK_DRIVER_PYTHON=$PWD/.venv310/bin/python
unset PYSPARK_PYTHON OPDI_WAREHOUSE
for d in 2026-06-05 2026-06-06 2026-06-07; do
  .venv310/bin/python -u benchmarks/run_adsc_day.py --day $d --arm both
done

# 2. Every figure on this page.
.venv310/bin/python -u benchmarks/regenerate_adsc_v1.py

# 3. This page.
cd ../opdi-portal/papers && quarto render adsc-fusion-v1/index.qmd
```

8 Provenance

Table 8.1: Each staged output, and the command that produced it. A `dirty` row was produced from uncommitted code: the number is real and cannot be reproduced from a commit.

Output	Script	Commit	Dirty tree
adsc_census.csv	benchmarks/adsc_census.py	d60bcc2	FALSE

Not staged: `adsc_gain_2026.csv`, `adsc_holes_2026.csv`, `adsc_time_sync_2026.csv`, `adsc_time_sync_pairs_2026.csv`, `adsc_events_2026.csv`, `adsc_events_by_airport_2026.csv`. The chapters above say so in place; this list is the index. A draft with visible holes is honest; a draft with numbers of unknown origin is not.